

Diplomová práce



České
vysoké
učení technické
v Praze

F3

Fakulta elektrotechnická
Katedra radioelektroniky

Embedded zařízení pro ukládání hesel

Embedded Password Storage Device

Bc. Jan Sedlák

Vedoucí práce: Ing. Ondřej Nentvich, Ph.D.
Studijní program: Technologie internetu věcí
Květen 2026

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Sedlák**

Jméno: **Jan**

Osobní číslo: **507277**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávací katedra/ústav: **Katedra radioelektroniky**

Studijní program: **Elektronika a komunikace**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Embedded zařízení pro ukládání hesel

Název bakalářské práce anglicky:

Embedded Password Storage Device

Pokyny pro vypracování:

Provedte rešerši možných variant zařízení pro ukládání dat, zejména se poté orientujte na ukládání přístupových údajů a jejich šifrování. Následně navrhnete a realizujete elektronické zařízení pro správu hesel a klíčů s využitím mikrokontroléru. Pro autorizaci přístupu ke správě záznamů využijte zadání uživatelského přístupového hesla nebo otisk prstu. Zadávání a předávání hesel s mikrokontrolérem realizujete s pomocí grafické aplikace.

Seznam doporučené literatury:

- [1] NOVIELLO, Carmine. Mastering STM32. 2nd. Leanpub, 2022.
- [2] KLEIDERMACHER, David a Mike KLEIDERMACHER. Embedded Systems Security. Elsevier, 2012. ISBN 978-0-12-386886-2.
- [3] MAREŠ, Martin a Tomáš VALLA. Průvodce labyrintem algoritmů. Praha: CZ.NIC, 2017. ISBN 978-80-88168-19-5.

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Ondřej Nentvich, Ph.D. katedra radioelektroniky FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **24.01.2024**

Termín odevzdání bakalářské práce: **24.05.2024**

Platnost zadání bakalářské práce: **21.09.2025**

Ing. Ondřej Nentvich, Ph.D.
podpis vedoucí(ho) práce

doc. Ing. Stanislav Vitek, Ph.D.
podpis vedoucí(ho) ústavu/katedry

prof. Mgr. Petr Páta, Ph.D.
podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Podpis studenta

TBD

Poděkování

Prohlášení

Prohlašuji, že jsem předloženou práci vypracoval samostatně a že jsem uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací.

V Praze, 16. dubna 2026

Abstrakt

TBD

Klíčová slova: embedded systém, heslo, otisk prstu, šifrování, mikrokontrolér, správce hesel

Abstract

TBD

Keywords: embedded system, password, fingerprint, encryption, microcontroller, password manager

Obsah

Úvod	1	2.7.2 Vizualizace	30
1 Teoretická část	3	3 Firmware	31
1.1 Heslo	3	3.1 Modul správy databáze	31
1.1.1 Anatomie hesla	3	3.2 Komunikační rozhraní	32
1.1.2 Délka: První úroveň ochrany	3	3.2.1 Návrh komunikačního protokolu	32
1.1.3 Komplexnost	3	3.3 Bezpečnostní modul a ochrana dat	33
1.1.4 Unikátnost	4	3.3.1 Autentizace uživatele	33
1.2 Správci hesel	4	3.3.2 Šifrování	34
1.2.1 Rozdělení Správců hesel	5	3.3.3 Distribuce a správa bezpečnostního klíče	34
1.2.2 Hardwarové řešení	7	3.4 Řídící algoritmus	35
1.3 Problematika zabezpečení embedded zařízení	7	3.5 Optimalizace a specifická řešení???	36
1.3.1 Secure boot	7	4 Desktopová aplikace	39
1.3.2 HSM (Hardware Security Module) vs TPM (Trusted Platform Module)	9	4.1 Architektura a datový model	39
1.3.3 TEE (Trusted Execution Environment)	9	4.2 Kybernetická bezpečnost	40
1.3.4 Code Signing	11	4.3 Uživatelské rozhraní	41
1.3.5 Ochrana paměti (MPU/MMU)	11	4.3.1 Connection tab	41
1.3.6 Bezpečné OTA aktualizace (Over-The-Air)	12	4.3.2 Databáze	41
1.4 Autentizace	13	4.3.3 Database tools tab	41
1.5 Šifrování	16	5 Ověření funkčnosti	45
1.6 Kybernetické útoky na embedded systémy	16	6 Závěr	47
1.6.1 Softwarové útoky	16	Zkratky	49
1.6.2 Síťové útoky	17	Literatura	51
1.6.3 Útok postraními kanály	18		
2 Návrh hardwaru	21		
2.1 Schéma zapojení	21		
2.2 Výběr kontroléru	22		
2.3 Power management	22		
2.3.1 Vstupní část	23		
2.3.2 Nabíjení baterie	23		
2.3.3 Návrh zdroje	24		
2.4 Výběr periférií	26		
2.4.1 Displej	26		
2.4.2 Senzor otisků prstů	27		
2.4.3 Rotační eknodér	28		
2.5 Návrh layoutu	29		
2.6 Výpočet spotřeby zařízení	30		
2.7 Konstrukční řešení a návrh krabičky	30		
2.7.1 Postup návrhu	30		

Obrázky

1.1 Mooltipass [22]	7
1.2 Trust Zone diagram [9]	10
2.1 Blokové schéma zapojení	21
2.2 Vybraný kontrolér ESP32 S3 WROOM 1	23
2.3 Zapojení kontroléru ESP32 S3 ..	23
2.4 Vstupní část power managementu	25
2.5 Nabíjecí obvod Li-Pol baterie ...	25
2.6 Navržený Buck-Boost zdroj	26
2.7 Vybrané komponenty	28
2.8 Layout navrženého PCB	29
2.9 Pohled shora	30
3.1 Master-Slave architektura	32
3.2 Otisk prstu	34
3.3 UI: Přidání nového otisku prstu	34
3.4 UI: Database	35
3.5 UI: Connect to Device	36
3.6 UI: Save Database	36
3.7 UI: Delete Database	36
3.8 UI: Device Info	37
3.9 Diagram logiky firmwaru	37
4.1 Struktura aplikace	40
4.2 Připojovací sekce aplikace	41
4.3 Databáze hesel	42
4.4 Nástroje	43
4.5 Funkce generování hesla	43
4.6 Uživatelské rozhraní aplikace ...	44

Tabulky

1.1 Příklady softwarových správců hesel	5
1.2 Přehled klíčových komponent Secure bootu	8
1.3 Porovnání HSM vs TPM	10
1.4 Typické použití TEE	11
1.5 Porovnání MPU vs MMU	13
2.1 Porovnání klíčových parametrů jednotlivých kontrolérů	24
2.2 Porovnání spínaného zdroje s LDO	26
2.3 Technické parametry vybraného displeje	27
2.4 Technické parametry senzoru DY50	27
2.5 Technické parametry eknodéru KY040	28
3.1 Placeholder Caption	35



Úvod

Heslo. Pojem, který zná každý z nás. Dnes a denně jej používáme a vytváříme nové. Přesto často nedbáme na jeho dostatečnou sílu a způsob uložení. Stáváme se tak jednoduchým cílem kybernetických útoků, a to ve světě, kde je digitální identita naprosto klíčová, může mít často fatální následky. Jedním z řešení je využít správce hesel, tedy nástroj, který umožňuje vytvářet silná hesla, bezpečně je ukládat a minimalizovat šanci na jejich odcizení.

Práce zkoumá jak možnosti bezpečného ukládání hesel, šifrování i hardwarové zabezpečení, tak i vybrané způsoby autentizace uživatele. Cílem této práce je provést rešerši aktuální nabídky nástrojů pro správu hesel a jejich šifrování a následně zrealizovat elektronické zařízení pro správu přístupových údajů s desktopovou aplikací.

Tato práce přináší nejen vhled do problematiky dnešní digitální bezpečnosti, ale také praktickou implementaci v podobě samotného zařízení. Jejím cílem je přispět k lepší ochraně digitální identity prostřednictvím uživatelsky přívětivých nástrojů pro správu hesel.

Kapitola 1

Teoretická část

V úvodu teoretické části se zaměřím, jak by mělo vypadat kvalitní, silné a neprolomitelné heslo, které má parametry odpovídající dnešní době. Tyto informace využiji pro porovnání jednotlivých řešení správců hesel. Následně rozeberu závažnost správného výběru šifrování, které podrobně analyzuji. V poslední části se zaměřím na autentizaci uživatele, kde na základě provedené rešerše přiblížím čtenáři nejbezpečnější způsoby ověření uživatele.

1.1 Heslo

Heslo je řetězec znaků, který slouží k ověření identity uživatele během jeho autentizace. Hesla se obvykle používají společně s dalším bezpečnostním prvkem, uživatelským jménem; jsou navržena tak, aby je znal pouze uživatel, kterému umožňují získat přístup k zařízení, aplikaci nebo webové stránce [?].

1.1.1 Anatomie hesla

Vytvoření neprolomitelného hesla vyžaduje mnohem více než jen kombinaci písmen a číslic. Pokud k jeho tvorbě chceme přistoupit zodpovědně, musíme brát v úvahu několik faktorů, které definují jeho sílu. Mezi hlavní faktory patří délka, komplexnost a unikátnost.

1.1.2 Délka: První úroveň ochrany

V době, kdy procesory dosahují neuvěřitelných rychlostí, hraje délka hesla klíčovou roli při určování jeho síly. Čím delší heslo, tím více existuje možných kombinací, což hackerům výrazně snižuje šanci na jeho prolomení. Minimální bezpečná délka je dnes uváděna jako 12 znaků, za předpokladu, že jsou použita malá a velká písmena, číslice a speciální symboly [1].

1.1.3 Komplexnost

Komplexnost je dalším, neméně důležitým faktorem, který dokáže ovlivnit sílu hesla a učinit z něj doslova neprolomitelný klíč. Uživatel by se měl vyvarovat čemukoliv, co ho s heslem byt jen nepatrně spojuje, jako jsou jména, oblíbené

sportovní týmy, důležitá a snadno dohledatelná data. Místo toho je vhodné použít zcela náhodná slova v kombinaci s čísly a symboly. Teprve tímto způsobem jsme schopni vytvořit neprolomitelné heslo [16].

1.1.4 Unikátnost

Používání stejného hesla pro několik různých přístupů je dnes nejčastější chybou, kterou dělá prakticky každý. Pramení z toho riziko, že pokud se útočníkovi podaří prolomit jedno vaše heslo, automaticky získá přístup i do ostatních účtů "chráněných" stejným heslem. Mít unikátní heslo pro každý jednotlivý účet se zdá být nemožným úkolem; přesto je však zásadním prvkem pro vaši bezpečnost. Přesně z tohoto důvodu je doporučeno používat správce hesel. Na trhu je jich nepřeberné množství a je tak pouze na uživateli, aby si vybral takového, který mu bude nejvíce vyhovovat [16].

PŘIDAT NĚJAKOU ZMÍNKU Z PRŮZKUMŮ, JAK LIDI PO-
UŽÍVAJÍ HESLA KOLIK JICH MAJÍ ATD

1.2 Správci hesel

Správce hesel je softwarový nebo hardwarový nástroj primárně navržený ke generování, uchovávání a správě přístupových údajů. Základním bezpečnostním principem většiny správců hesel je šifrování databáze s využitím silných symetrických šifer (standardně AES-256). K odemčení této databáze používá uživatel hlavní heslo.

Hlavní heslo se nikdy neukládá v prostém textu; místo toho se z něj pomocí funkcí pro derivaci klíče (KDF – Key Derivation Function, typicky PBKDF2, Argon2 nebo bcrypt) vygeneruje šifrovací klíč. Národní institut standardů a technologie (NIST) ve svém doporučení SP 800-63B výslovně podporuje používání správců hesel, neboť eliminují nutnost pamatovat si velké množství komplexních hesel a minimalizují riziko jejich recyklace [6].

Moderní správci hesel se z prostých šifrovaných databází vyvinuli v komplexní nástroje pro správu digitální identity. Kromě samotného bezpečného uložení přihlašovacích údajů nabízejí sadu funkcí, které zvyšují jak bezpečnost, tak uživatelskou přívětivost:

- **Generování kryptograficky silných hesel** - Schopnost automaticky vytvářet hesla s vysokou entropií.
- **Automatické vyplňování** - Nástroj detekuje doménu webové stránky a automaticky vyplní přihlašovací údaje. Kromě pohodlí jde o klíčovou ochranu proti phishingu – pokud uživatel klikne na podvodný odkaz (např. g00gle.com), správce hesel pole nevyplní, protože doména neodpovídá uloženému záznamu.
- **Bezpečné sdílení** - Možnost asymetricky zašifrovat konkrétní záznam a bezpečně jej sdílet s jiným uživatelem (např. v rámci rodiny nebo firmy).

aniž by heslo bylo odesláno přes nezabezpečené kanály, jako je e-mail nebo chat.

- **Podpora vícefaktorové autentizace (MFA) a správa TOTP** - Většina správců hesel dnes umí generovat jednorázová hesla (Time-based One-Time Password), čímž nahrazuje dedikované aplikace jako Google Authenticator.
- **Monitoring kompromitovaných hesel a Dark Webu** - Nástroje automaticky porovnávají hashe uložených hesel s databázemi uniklých přihlašovacích údajů (např. služba Have I Been Pwned) a aktivně uživatele upozorňují na nutnost změny.
- **Nouzový přístup** - Funkce umožňující delegovat přístup do trezoru pověřené osobě po uplynutí předem stanovené čekací doby (např. v případě nehody či úmrtí).

1.2.1 Rozdělení Správců hesel

Správce hesel lze rozdělit z hlediska samotné implementace a způsobu uložení dat na dvě hlavní kategorie - softwarová a hardwarová řešení.

Softwarová řešení

Softwaroví správci hesel existují nejčastěji jako samostatné aplikace pro operační systémy nebo jako rozšíření pro webové prohlížeče. Dále se dělí na **cloudové** (data jsou synchronizována přes servery poskytovatele, např. Bitwarden, 1Password), **lokální** (data neopouštějí zařízení, např. KeePass) a **správce v rámci prohlížečů** (Google Chrome, Mozilla Firefox a další).

Typ	Název
Lokální	KeePassXC
Lokální	Valutwarden
Lokální	Enpass
Cloudový	Bitwarden
Cloudový	Dashline
Cloudový	Proton Pass

Tabulka 1.1: Příklady softwarových správců hesel

Lokální správci hesel

U lokálních správců (např. KeePassXC, Password Safe) je zašifrovaná databáze uložena výhradně na fyzickém úložišti uživatele a nikdy není automaticky přenášena přes internet. Mezi hlavní výhody patří naprostá kontrola nad daty. Tím, že data neopouštějí zařízení, útočník bez fyzického přístupu k zařízení nebo bez infikování stroje malwarem je nemůže získat. Nic není dokonalé a i lokální řešení přináší svá rizika:

- **Ztráta a fragmentace dat** - Pokud uživatel nebude svá data průběžně zálohovat a dojde k selhání hardwaru, je ztráta dat nevratná. Synchronizace napříč zařízeními (např. přes USB nebo sdílený disk) často vede ke kolizím verzí databáze.
- **Forenzní a paměťová zranitelnost** - Jak ve své studii prokázali *Gray et al. (2016)*, lokální správci hesel mohou být zranitelní vůči analýze operační paměti (RAM). Výzkum ukázal, že po odemčení databáze nebo dokonce po uzavření aplikace mohou fragmenty hlavního hesla či dešifrovaných dat zůstat v paměťových stránkách, odkládacích souborech (pagefile) nebo dočasných složkách, odkud je může sofistikovaný malware extrahovat [7].

Cloudoví správci hesel

Tato řešení (např. Bitwarden, 1Password) ukládají zašifrované trezory na centrálních serverech poskytovatele. Využívají tzv. architekturu nulové znalosti (Zero-Knowledge architecture), což znamená, že veškeré šifrování a dešifrování probíhá lokálně na koncovém zařízení uživatele a na server se odesílá pouze zašifrovaný řetězec dat. Hlavní výhodou je dokonalá a okamžitá synchronizace napříč všemi platformami, na kterých je uživatel přihlášen, a automatické zálohování zajištěné poskytovatelem. Stejně jako lokální řešení, ani cloudová správa se neobejdou bez rizik:

- **Hromadné úniky dat ze strany poskytovatele** - Servery poskytovatelů představují vysoce atraktivní cíl (tzv. honeypot) pro kyberzločince. Pokud útočníci prolomí infrastrukturu poskytovatele, mohou zkopírovat zašifrované trezory všech uživatelů. Zde vzniká riziko offline dictionary útoku: pokud má uživatel slabé hlavní heslo, útočník jej může na vlastním výkonném hardwaru bez omezení počtu pokusů prolomit.
- **Závislost na infrastruktuře třetí strany** - Výpadek serverů může dočasně zamezit přidávání či synchronizaci nových hesel.
- **Útoky na dodavatelský řetězec (Supply Chain)** - Zneužití aktualizčního mechanismu nebo zkompromitování webového klienta může vést k podvržení škodlivého kódu (např. JavaScriptu), který by hlavní heslo zachytil dříve, než dojde k lokálnímu zašifrování.

Správci hesel v prohlížečích

Prohlížeče jako Google Chrome, Mozilla Firefox nebo Microsoft Edge dnes obsahují nativní správce hesel. Jsou hluboce integrovány do webového rozhraní a spjatý s uživatelským profilem prohlížeče. Hlavní výhodou je jednoduchost vstupu pro běžného uživatele. Nevyžadují instalaci žádného dodatečného softwaru a fungují plynule z pohledu uživatelského zážitku (UX). Je tedy zcela na uživateli, zda se pro jeho užívání rozhodne. Oproti předchozím řešením mívají tito správci omezené funkce. Velkým rizikem použití mohou být podvržené webové stránky s neviditelným formulářem, který zneužívá funkci automatického vyplnění údajů.

1.2.2 Hardwarové řešení

Fyzická řešení představují oddělená hardwarová zařízení navržená tak, aby chránila šifrovací klíče a hesla mimo běžný operační systém. Patří sem specializovaní hardwaroví správci hesel a bezpečnostní tokeny (např. YubiKey, Trezor), které k autentizaci využívají protokoly FIDO2, WebAuthn nebo UWB (Ultra-Wideband) lokalizaci pro potvrzení fyzické přítomnosti [3]. Na trhu již existovalo několik typů embedded správců hesel, žádný z nich se však neuchytil v běžném každodenním používání.

Za zmínku stojí například *PasswordFast* a *PasswordSafe*, obě zařízení disponovala klávesnicí a displejem, nicméně pro každodenní používání byla nepraktická. Nejkomplexnějším řešením bylo zařízení *Mooltipass*, které bylo zabezpečeno hlavním heslem a šifrováním AES-256. Umožňovalo připojení k libovolnému zařízení pomocí bluetooth a následné doplňování údajů do přihlašovacích formulářů. Pozdější verze disponovala i rozšířením do prohlížeče.



Obrázek 1.1: Mooltipass [22]

1.3 Problematika zabezpečení embedded zařízení

Zabezpečení embedded zařízení se v posledních letech stalo kritickou disciplínou, především kvůli masivnímu rozvoji Internetu věcí (IoT). Na rozdíl od běžných počítačů jsou embedded systémy často nasazeny v nekontrolovaném prostředí, což je vystavuje nejen síťovým, ale i přímým fyzickým útokům. Z tohoto důvodu je třeba myslet na zabezpečení systému již v samotném počátku jeho návrhu. V této sekci bude rozebráno několik softwarových i hardwarových způsobů ochrany, které se používají pro embedded systémy.

1.3.1 Secure boot

Secure Boot je kritický bezpečnostní mechanismus, který zajišťuje, aby se v zařízení spustil pouze software, který je autentický a nebyl nijak pozměněn. V prostředí embedded systémů se jedná o nejdůležitější prvek ochrany, protože tvoří tzv. Root of Trust, na kterém stojí veškeré další zabezpečení [11].

operačního systému. Zároveň znemožňuje útočníkovi nahrát vlastní, upravený firmware či rovnou vyměnit flash paměť s kódem. V takovém případě by hash nesouhlasil.

Použití secure bootu přináší i svá úskalí v podobě ztráty klíčů. Pokud výrobce ztratí soukromé klíče, je nemožné vydat aktualizaci pro existující zařízení. V případě výskytu chyby v samotném RoT je chyba uložena v ROM paměti a daný hardware je tak trvale a neopravitelně kompromitován [21].

■ 1.3.2 HSM (Hardware Security Module) vs TPM (Trusted Platform Module)

■ Hardware Security Module

Zatímco TPM je zaměřen na integritu platformy, HSM je vysoce výkonné zařízení zaměřené na správu životního cyklu klíčů a masivní kryptografické operace. V embedded světě se často setkáváme s Hardware Security Engine (HSE) nebo Secure Hardware Extension (SHE), což jsou "lehčí" verze HSM integrované přímo v mikrokontrolérech (např. v automotive průmyslu) [14].

HSM jsou optimalizovány pro rychlé zpracování kryptografických operací, jako jsou AES, RSA nebo ECC, bez zatěžování hlavního procesoru.

Samotné HSM běží na vlastním kódu, který je zcela nezávislý na hlavním firmwaru zařízení. Při proniknutí útočníka do hlavního systému tak zůstává HSM neovládnut [4].

■ Trusted Platform Module

TPM je mezinárodní standard pro bezpečný kryptoprocessor. V embedded systémech se nejčastěji vyskytuje jako samostatný čip připojený přes sběrnici (SPI, I2C) nebo jako integrovaná součást SoC (System on Chip) [14].

TPM dokáže hashovat stav systému během bootování, a pokud dojde ke změně firmwaru nebo jeho konfigurace, hash v konfiguračních registrech nebude odpovídat a TPM odmítne vydat dešifrovací klíč.

Disponuje bezpečným uložištěm pro ukládání hesel, certifikátů a šifrovacích klíčů. Klíče jsou označeny jako neexportovatelné a nikdy neopustí čip v čitelné podobě. Pro dosažení silné kryptografie disponuje TPM také hardwarovým generátorem náhodných čísel [4].

■ 1.3.3 TEE (Trusted Execution Environment)

Trusted Execution Environment (TEE) je bezpečné, izolované prostředí uvnitř hlavního procesoru, které běží paralelně s hlavním operačním systémem. Toto

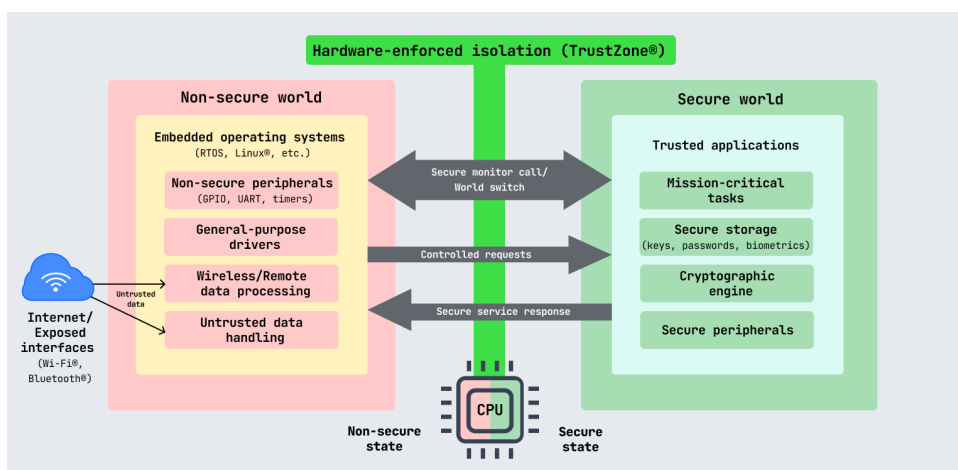
Tabulka 1.3: Porovnání HSM vs TPM

Funkce	TPM (Trusted Platform Module)	HSM (Hardware Security Module)
Hlavní účel	Integrita zařízení a bezpečný boot.	Rychlá kryptografie a ochrana klíčů.
Výkon	Nižší (vhodné pro autentizaci).	Vysoký (vhodné pro šifrování streamů dat).
Typ klíčů	Často pevně vázané na hardware.	Flexibilní správa tisíců klíčů.
Standardy	TCG (Trusted Computing Group).	FIPS 140-2 (Level 3/4), Common Criteria.
Typické použití	Notebooky, průmyslová PC, brány.	Bankovní terminály, servery, řídicí jednotky aut.

prostředí je hardwarově odděleno od zbytku procesoru. Zatímco hlavní operační systém je optimalizován pro výkon a konektivitu, TEE je navržen s prioritou a optimalizací pro bezpečnost a integritu dat [15].

Nejrozšířenější implementací TEE je technologie *ARM TrustZone*. Ta rozděluje procesor na dvě virtuální domény:

- **Normal World (Non-secure)** - Zde běží běžný OS (např. Linux, Android) a uživatelské aplikace. Přístup do *Secure world* je striktně omezen.
- **Secure World** - Zde běží veškerý *Trusted OS* a v něm izolované mikro-aplikace, známé jako *Trusted Applications*.

**Obrázek 1.2:** Trust Zone diagram [9]

Z obrázku 1.2 lze vidět, jakým způsobem probíhá komunikace mezi oběma částmi procesoru. Z nezabezpečeného světa vyšle operační systém kontrolo-

vaný dotaz, *Controlled request*, v tento okamžik je aktivován i *Secure monitor*, který dohlíží na veškerou komunikaci. Po zpracování požadavku odesílá *Trusted aplikace* odpověď, neboli *Secure Service Response*. S odesláním odpovědi se *secure monitor* opět vypíná.

TEE poskytuje tři základní pilíře ochrany. Prvním je *izolace*. Jak bylo již napsáno, hardwarová izolace v rámci samotného SoC zajistí, že ani při ovládnutí jádra operačního systému nemůže útočník číst paměť TEE nebo do ní zapisovat. Druhým pilířem je důvěryhodné uložení, *Trusted storage*, kdy TEE má přístup do zašifrované části paměti, kam se ukládají citlivá data, jako jsou otisky prstů či privátní klíče. Posledním pilířem je *vzdálená atestace*, což je schopnost prokázat vzdálené protistraně, že zařízení běží v autentickém a bezpečném stavu. V následující tabulce 1.4 naleznete přehled typického použití TEE.

Tabulka 1.4: Typické použití TEE

Oblast	Využití v TEE
Biometrika	Zpracování a porovnávání otisků prstů nebo skenů obličeje (data nikdy nevidí Android/Linux).
Finanční transakce	Bezpečné potvrzení platby a ochrana PINu při zadávání na obrazovce (Trusted UI).
DRM (Digital Rights Management)	Dešifrování prémiového obsahu (např. Netflix v 4K) přímo v bezpečném prostředí, aby nebylo možné stream zachytit.
Kryptografické operace	Podepisování transakcí pomocí klíčů, které nejsou přístupné hlavnímu procesoru.

1.3.4 Code Signing

Code Signing slouží v embedded systémech jako ochranná pečeť pro samotný zdrojový kód. Je používán jako nástroj pro samotný Secure Boot mechanismus zaručující *autenticitu* a *integritu* firmwaru [11].

1.3.5 Ochrana paměti (MPU/MMU)

V systémech s omezenými zdroji je paměť často sdíleným prostředkem. Bez hardwarové ochrany může jediná chyba v aplikaci vést ke zhroucení celého systému nebo jeho zneužití útočníkem pro injekci malwaru. Mezi nejčastější chyby patří například přetečení bufferu paměti nebo chybný ukazatel proměnné. K eliminaci těchto rizik slouží dedikované hardwarové jednotky integrované v procesoru.

■ Memory Protection unit (MPU)

Memory protection unit je zjednodušená verze ochrany paměti, která se nachází především v mikrokontrolérech, například ARM Cortex-M. MPU nepracuje s virtuální pamětí, ale přímo s fyzickými adresami.

Paměť je rozdělena do několika fixních částí, typicky 8 až 16. Každá část má programátorem definovanou počáteční adresu, velikost a přístupová práva. Každá část může disponovat různými režimy přístupu.

- Read-Only (RO)
- Read-Write (RW)
- Execute-Never (XN)

Režim Execute-Never je kritickým režimem pro zabezpečení paměti, jelikož zabráňuje procesoru vykonávat kód z datových oblastí, jako je zásobník/stack. Tento režim funguje jako primární obrana proti útokům typu Stack Overflow [23].

V případě, že se kód pokusí přistoupit do nepovoleného regionu paměti, MPU vyvolá hardwarovou výjimku, která okamžitě zastaví proces a umožní systému bezpečně zareagovat.

■ Memory Management Unit (MMU)

Memory Management Unit je pokročilejší jednotka využívaná v aplikačních procesorech, například ARM Cortex-A, které provozují komplexní operační systémy, jako jsou Linux nebo QNX.

Hlavním rozdílem oproti MPU je podpora virtuálního adresního prostoru. Každý proces má pocit, že vlastní celý paměťový prostor, zatímco MMU mapuje tyto virtuální adresy na reálné fyzické adresy pomocí stránkování (paging) [19]. Díky MMU jsou jednotlivé procesy dokonale izolované a jeden proces fyzicky nemůže vidět do paměti druhého procesu, pokud není nastavená sdílená paměť.

MMU úzce spolupracuje s úrovněmi oprávnění procesoru, user vs. kernel mode. To zajišťuje, že aplikace v uživatelském rozhraní nemohou měnit nastavení hardwaru nebo přistupovat k paměti jádra [11].

■ 1.3.6 Bezpečné OTA aktualizace (Over-The-Air)

U moderních IoT zařízení je kritickou funkcí Over the Air (OTA) aktualizace. V praxi to znamená, že zařízení je možné bezpečně aktualizovat bez nutnosti fyzického připojení. Aktualizace je považována za bezpečnou, je-li zdrojový kód opatřen výše zmiňovaným *Code signem*, v případě jeho absence by útočník mohl podvrhnout vlastní škodlivý firmware, například pomocí útoku

Tabulka 1.5: Porovnání MPU vs MMU

Funkce	MPU	MMU
Typ zařízení	Mikrokontroléry (RTOS)	Aplikační procesory (Linux/Android)
Adresování	Pouze fyzické adresy	Virtuální adresování
Granularita	Velké regiony	Malé stránky (typicky 4 KB)
Hlavní přínos	Prevence chyb (Safety)	Plná izolace procesů (Security)

Man in the Middle, který by mu zajistil plnou kontrolu nad zařízením a volný přístup do celého systému. Standardy jako *NIST SP 800-147*¹ striktně vyžadují, aby každý update prošel verifikací dříve, než dojde k jeho zápisu do flash paměti zařízení [5].

1.4 Autentizace

Autentizace je jedním ze základních pilířů informační bezpečnosti. Jedná se o proces ověření deklarované identity subjektu (typicky uživatele), který žádá o přístup k chráněným zdrojům nebo systémům. Na rozdíl od identifikace, která pouze určuje, o koho se jedná, autentizace prokazuje, že daný subjekt je skutečně tím, za koho se vydává[20].

V moderních informačních systémech se autentizační metody dělí primárně do tří kategorií (tzv. autentizačních faktorů)[13]:

- **Znalostní faktor** - Něco, co uživatel zná - heslo, pin.
- **Faktor vlastnictví** - Něco, co uživatel vlastní - hardwarový token, čipová karta, mobilní telefon.
- **Biometrický faktor** - Něco, čím uživatel skutečně je - otisk prstu, geometrie obličeje.

Tato kapitola se zaměřuje na detailní rozbor vybraných znalostních a biometrických metod, konkrétně na využití hlavního hesla, PIN kódu a na autentizaci pomocí otisku prstu, s důrazem na princip fungování optického senzoru, který byl využit pro zhotovení prototypu embedded správce hesel.

Znalostní metody autentizace Znalostní faktory jsou historicky nejstarším a nejrozšířenějším způsobem ověřování identity. Jejich princip spočívá v tom, že autentizační systém i uživatel sdílejí určité tajemství.

Hlavní heslo

Heslo je řetězec znaků, který slouží k přístupu do systému. V kontextu centralizovaných nebo komplexních systémů se často využívá koncept tzv. hlavního hesla (Master Password), které odemýká přístup k dalším dílčím heslům nebo kryptografickým klíčům (např. u správců hesel)[2]. Jak se tvoří neprolomitelné

¹<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-147.pdf>

Princip fungování optického senzoru

Pro sejmutí obrazu otisku prstu se využívá několik technologií (kapacitní, ultrazvukové, optické). Optické senzory patří k nejstarším, avšak díky inovacím (např. integraci pod displeje mobilních telefonů) zůstávají stále velmi aktuálními technologiemi.

Optický senzor funguje v zásadě jako specializovaný digitální fotoaparát. Skládá se ze tří hlavních komponent:

- Zdroj světla, typicky pole LED diod.
- Dotyková plocha / hranol, kam se přikládá prst.
- Snímací čip

Fyzikální princip snímání (FTIR) Většina moderních optických senzorů využívá fyzikální jev zvaný narušený úplný vnitřní odraz (FTIR – Frustrated Total Internal Reflection). Když světlo ze zdroje vstoupí do skleněného hranolu pod určitým úhlem, dojde na rozhraní skla a vzduchu (kde není přiložen prst) k úplnému vnitřnímu odrazu. Světlo se odrazí zpět do nitra hranolu a dopadne na snímací čip.

Když uživatel přiloží prst na sklo, nedotýká se plochy celou kůží, ale pouze hřebeny papilárních linií. V místech, kde se hřebeny (ridges) dotýkají skla, se změní index lomu světla na rozhraní. Úplný vnitřní odraz je v těchto bodech "narušen" – světlo je absorbováno nebo rozptýleno do kůže prstu. Naopak v místech údolí (valleys) mezi papilárními liniemi zůstává mezi sklem a kůží mikroskopická vrstva vzduchu, a proto zde k úplnému vnitřnímu odrazu nadále dochází[12].

Výsledkem je, že snímací CMOS/CCD čip zaznamená vysoce kontrastní 2D obraz:

- **Tmavé oblasti** - Představují hřebeny papilárních linií (světlo se rozptýlilo do prstu).
- **Světlé oblasti** - Představují údolí a místa bez kontaktu (světlo se odrazilo do senzoru).

Tento obraz je následně převeden do digitální podoby, vyčištěn od šumu, zvýrazněn a předán softwarovým algoritmům k extrakci výše zmíněných markantů. Nevýhodou základních optických senzorů je jejich náchylnost k oklamání pomocí kvalitních 2D fotografií nebo 3D odlitků prstů, proto jsou dnešní systémy často doplňovány o techniky detekce živosti (liveness detection), jako je měření pulsu nebo prokrvení tkáně.

1.5 Šifrování

1.6 Kybernetické útoky na embedded systémy

Dnešní svět je plný včestavěných systémů, od jednoduchých spotřebičů, jako jsou mikrovlnné trouby, až po chytrá auta. Embedded systémy do značné míry ulehčují naše životy; na druhou stranu však přidávají další možnosti pro útočníky, jak se zmocnit našeho soukromí.

Kybernetická bezpečnost je i v tomto odvětví velice důležitá. Valná většina chytrých systémů dnes generuje, přenáší a zpracovává signifikantní množství dat, která jsou pro hackery lákadlem.

Embedded systémy jsou náchylné na kybernetické útoky hned z několika důvodů. Prvním z nich je existence dvou zranitelných oblastí, a to jak na hardwarové, tak na softwarové úrovni. Dalším důvodem je, že přeplnění malého včestavěného systému mnoha funkcemi vede k nedostatečné bezpečnostní koncepci. Navíc integrace včestavěných systémů do internetu věcí (IoT) zvyšuje počet útočných vektorů, které by se mohly zaměřit právě na tyto systémy[10].

1.6.1 Softwarové útoky

1. Útoky na paměť (Memory Corruption)

Většina embedded systémů je psána v jazycích C nebo C++, které poskytují přímý přístup k paměti, ale postrádají automatickou ochranu.

- **Buffer Overflow (Přetečení zásobníku/haldy)** - Útočník odešle více dat, než kolik dokáže přidělená paměť pojmout. Přetečená data mohou přepsat návratovou adresu funkce, čímž útočník získá kontrolu nad tokem programu.
- **Arbitrary Code Execution (ACE)** - Cíl útoku na paměť. Útočník do systému "podstrčí" vlastní kód (shellcode) a donutí procesor, aby jej vykonával.
- **Return-Oriented Programming (ROP)** - Moderní technika, která obchází ochranu proti spuštění kódu v datových segmentech tím, že skládá útok z malých částí legitimního kódu (gadgets) již přítomného v paměti.

2. Zranitelnost komunikačních protokolů

- **Injekce (Injection Attacks)** - Pokud systém nedostatečně filtruje vstup, může útočník vložit škodlivé příkazy (např. SQL injection u bran, nebo Command Injection v CLI rozhraní).

- **Replay Attack (Útok opakováním)** - Útočník zachytí legitimní zprávu (např. signál k odemknutí chytrého zámku) a později ji znovu odešle bez nutnosti znát šifrovací klíč.
- **Man-in-the-Middle (MitM)** - Útočník se vloží mezi zařízení a cloud/bránu. Pokud chybí autentizace certifikátů, může data číst i měnit.

3. Útoky na firmware a bootloader

- **Firmware Downgrade** - Útočník donutí zařízení nainstalovat starší verzi firmwaru, která obsahuje známé a již dříve opravené bezpečnostní chyby.
- **Úprava Update balíčků** - Pokud firmware není kryptograficky podepsán, může útočník nahrát upravenou verzi (backdoor), která mu zajistí trvalý přístup.
- **Secure Boot Bypass** - Softwarová chyba v bootloaderu, která umožní spustit nepodepsaný kód hned při startu systému.

Prevence a obrana

Obrana proti těmto útokům zahrnuje techniky, jako jsou Secure Boot, použití ASLR (Address Space Layout Randomization), pravidelné podepsané aktualizace (OTA) a statickou analýzu kódu během vývoje.

1.6.2 Síťové útoky

1. Útoky na komunikační protokoly

Mnoho embedded zařízení používá protokoly, které byly navrženy pro efektivitu, nikoliv pro bezpečnost (např. MQTT, CoAP, Modbus).

- **Replay Attack** - Útočník zachytí platný síťový paket (např. příkaz "odemknout zámek") a později jej odešle znovu. Pokud systém nemá implementované sekvenční číslování nebo časová razítka, příkaz provede znovu.
- **Protocol Fuzzing** - Odesílání neočekávaných nebo poškozených datových paketů do síťového stacku zařízení s cílem vyvolat pád systému (DoS) nebo odhalit zranitelnosti typu přetečení zásobníku (buffer overflow)

2. Denial of Service (DoS) a Resource Exhaustion

Embedded systémy mají často velmi limitovanou RAM a procesorový výkon.

- **Flooding** - I malý objem provozu (např. ICMP/Ping flood) může zahltit síťové rozhraní mikrokontroléru, který nestíhá pakety zahazovat.

1. Časová analýza

Časová analýza zkoumá dobu potřebnou k provedení určitých kryptografických operací. Pokud doba zpracování závisí na vstupních datech nebo na tajném klíči, může útočník tyto citlivé údaje odvodit.

- **Příklad:** Pokud algoritmus ověřující heslo (nebo PIN) kontroluje znaky postupně a při první neshodě proces ukončí, zabere zamítnutí hesla s chybným prvním znakem méně času než zamítnutí hesla, kde je chyba až ve třetím znaku. Útočník tak může uhodnout správné heslo znak po znaku.

2. Odběrová analýza (Power Analysis)

Každá operace prováděná mikroprocesorem nebo mikrokontrolérem spotřebovává určité množství elektrické energie. Spotřeba se mění v závislosti na tom, jaké instrukce se právě vykonávají a jaká data jsou zpracovávána (např. přepínání bitů z 0 na 1 vyžaduje více energie).

- **Simple Power Analysis** - Spočívá v přímém vizuálním zkoumání grafu spotřeby energie v čase. Umožňuje identifikovat hrubé rysy algoritmu, například rozeznat podmíněné skoky, které závisí na hodnotách tajného klíče.
- **Differential Power Analysis** - Mnohem sofistikovanější a nebezpečnější metoda. Využívá statistickou analýzu velkého množství měření (tisíce až miliony vzorků) spotřeby energie. DPA dokáže odhalit korelaci mezi spotřebovanou energií a manipulovanými daty, čímž útočník získá tajný klíč i v případě, že je signál silně zarušen.

3. Elektromagnetická analýza (Electromagnetic Analysis)

Místo spotřeby elektrického proudu útočník měří elektromagnetické vyzařování, které zařízení generuje během provozu.

- **Výhoda pro útočníka** - Pomocí citlivých mikrosond může útočník měřit vyzařování lokálně – přímo nad konkrétní částí čipu (například nad kryptografickým koprocesorem), čímž se vyhne šumu z ostatních komponent zařízení. Metody analýzy (SEMA, DEMA) jsou analogické k analýze spotřeby (SPA, DPA).

4. Akustická a optická analýza

Ačkoliv jsou tyto útoky u běžných mikrokontrolérů vzácnější, některé elektronické součástky (např. kondenzátory nebo cívky) mohou při zátěži vydávat vysokofrekvenční zvuky. Měřením tohoto akustického signálu lze teoreticky odvodit informace o prováděných instrukcích. Podobně může vyzařování fotonů z polovodičových přechodů (optický kanál) prozradit stavy tranzistorů, to však vyžaduje speciální vybavení a odkrytý čip.

Obrana proti útokům postranními kanály

Návrh vestavěných systémů odolných proti SCA vyžaduje kombinaci hardwarových a softwarových protiopatření. Cílem je buď eliminovat závislost

Kapitola 2

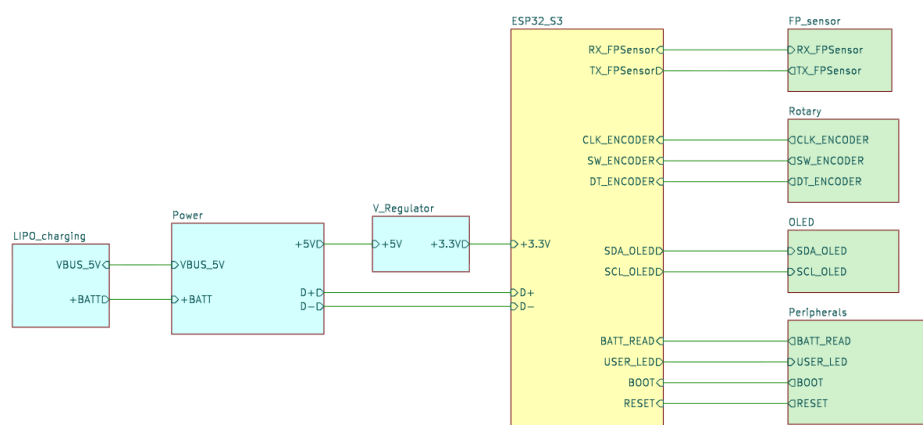
Návrh hardwaru

Tato kapitola se zabývá praktickým návrhem hardwarové části mé diplomové práce. Samotný návrh vychází z teoretických poznatků a specifikace požadavků, které byly definovány v teoretické části této práce 1. Cílem této části je vytvořit funkční prototyp, který bude splňovat požadavky na kompaktní rozměry, nízkou spotřebu a vlastní napájení.

Ještě před samotným návrhem jsem sestrojil funkční prototyp z jednotlivých vývojových kitů, které mi pomohly při prvotním návrhu.

2.1 Schéma zapojení

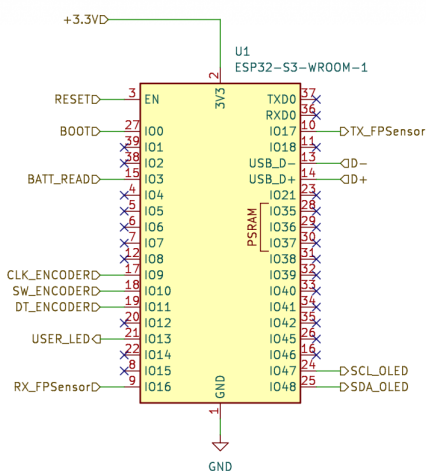
Na následujícím obrázku 2.1 je blokové schéma navrženého prototypu zařízení. Samotné schéma se skládá ze tří částí: kontrolér(žlutá), power management(modrá) a periférie(zelená).



Obrázek 2.1: Blokové schéma zapojení



Obrázek 2.2: Vybraný kontrolér ESP32 S3 WROOM 1



Obrázek 2.3: Zapojení kontroléru ESP32 S3

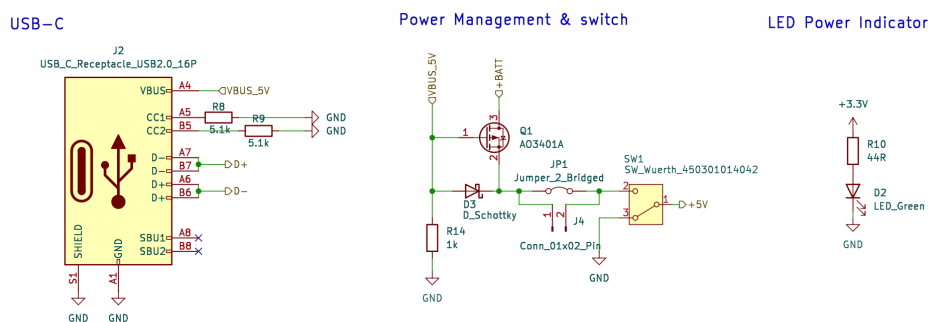
2.3.1 Vstupní část

Vstupní napájecí rozhraní tvoří konektor USB-C 2.0. Pro identifikaci napájecího zdroje jsou na pinech CC1 a CC2 připojeny rezistory $5,1\text{ k}\Omega$. Automatická volba zdroje je zaručena použitím P-kanálového MOSFET tranzistoru (Q1) v kombinaci se Schottkyho diodou (D3).

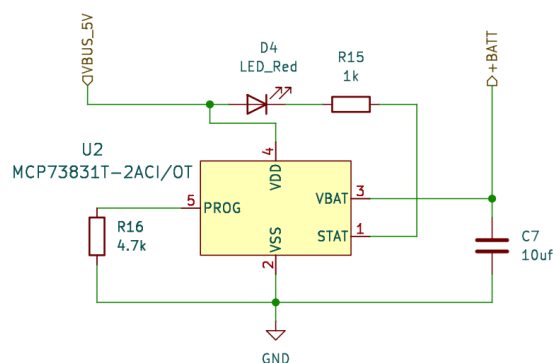
- **USB:** Napětí na hradle uzavře tranzistor Q1, čímž odpojí baterii a napájí se přes diodu D3.
- **Baterie:** Po odpojení USB rezistor R14 stáhne hradlo k zemi, tranzistor Q1 se otevře a zařízení se začne napájet z baterie.

2.3.2 Nabíjení baterie

Pro správu nabíjení bateriového článku byl zvolen specializovaný lineární kontrolér MCP73831T-2ACI/OT. Obvod zajišťuje nabíjecí cyklus typu CC/CV (Constant Current / Constant Voltage). Nabíjecí proud je definován hodnotou odporu $R16 = 4,7\text{ k}\Omega$ připojeného k pinu PROG. Hodnota proudu odpovídá



Obrázek 2.4: Vstupní část power managementu



Obrázek 2.5: Nabíjecí obvod Li-Pol baterie

Low-dropout regulátor

LDO je lineární stabilizátor napětí, který dokáže udržovat konstantní úroveň napětí i v případě, že je vstupní napětí pouze velmi mírně vyšší než výstupní. LDO mění přebytečnou energii na teplo, tím se snižuje i celková výsledná účinnost regulátoru. Výhodou tohoto zdroje je jeho jednoduchost, základními komponenty jsou vstupní a výstupní kondenzátory spolu s integrovaným obvodem.

Spínaný zdroj

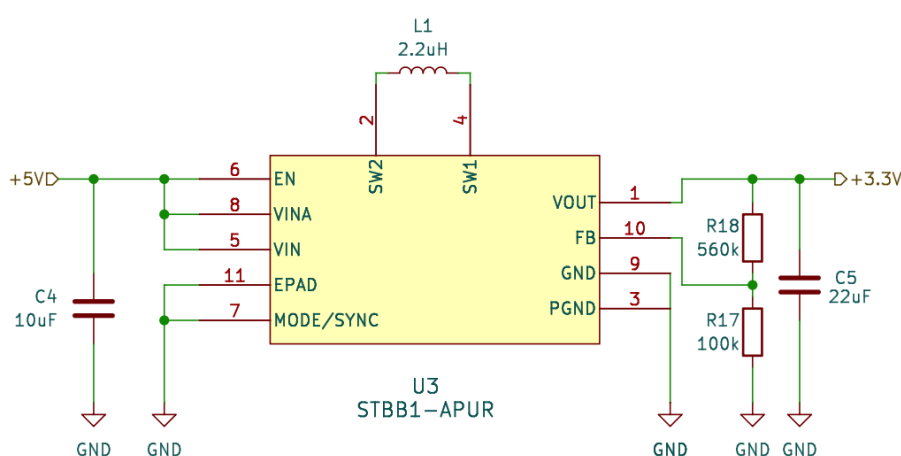
Na rozdíl od LDO regulátoru spínaný zdroj funguje na principu rychlého přepínání výkonového prvku, MOSFET tranzistoru. Spínaný zdroj tranzistor spíná v řádech desítek kHz až MHz , proto se doporučuje jeho umístění do oddělené části PCB, aby se zabránilo rušení. Spínaný zdroj dosahuje oproti LDO vyšší účinnosti. Nevýhodou spínaného zdroje je jeho robustnost. Vyžaduje více základních komponentů a stává se tak dražší volbou.

V následující tabulce 2.2 je porovnání mezi vybraným zdrojem STBB1 od společnosti STMicroelectronics a LDO regulátorem AMS1117, který se velmi často používá pro návrh mikrokontrolérů ESP32.

Tabulka 2.2: Porovnání spínaného zdroje s LDO

Vlastnosť	Spínaný zdroj (STBB1)	Lineárny regulátor (AMS1117)
Účinnosť	Vysoká (typ. 85–95 %)	Nízka (66 %)
Tepelné ztráty	Minimálne	Vysoké
Využití baterie	Úplné	Částečné
Šum a rušení	Vyšší	Veľmi nízke

Z výše uvedených informací je zdánlivé, že použití spínaného zdroje bude výhodnější.



Obrázek 2.6: Navržený Buck-Boost zdroj

2.4 Výběr periferií

Na základě sestavení prvotního prototypu a výběru mikrokontroléru ESP32-S3 se tato sekce věnuje výběru jednotlivých hardwarových periférií. Hlavním kritériem při procesu selekce byla vzájemná napěťová kompatibilita na úrovni 3,3 V logiky a kompatibilita s dostupnými softwarovými knihovnami.

V následujících podsekcích jsou detailně popsány klíčové periferie, včetně odůvodnění jejich volby.

■ 2.4.1 Displej

Pro uživatelské rozhraní byl zvolen OLED displej s úhlopříčkou 1,3" a rozlišením 128×64 pixelů, který je založen na řadiči SH1106. Hlavním důvodem volby technologie OLED je jeho vysoký kontrastní poměr. Jelikož displej nevyžaduje podsvícení, výrazně se tak snižuje jeho energetická náročnost.

Komunikace s mikrokontrolérem probíhá přes sběrnici I2C, což minimalizuje počet potřebných pinů kontroléru a zjednodušuje návrh plošného spoje.

Tabulka 2.3: Technické parametry vybraného displeje

Parametr	Hodnota
Typ displeje	OLED
Řadič (Driver)	SH1106
Rozlišení	128 × 64 px
Úhlopříčka	1,3" (33,02 mm)
Vnitřní RAM řadiče	132 × 64 bit
Komunikační rozhraní	I2C
Napájecí napětí	3,3 – 5,0 V DC

2.4.2 Senzor otisků prstů

Jako biometrický bezpečnostní prvek byl vybrán otisk prstu. Pro jednoduchost byl vybrán senzor DY50, který je jedním z mála dostupných modelů za přijatelnou cenu. Jedná se o senzor s integrovaným optickým snímacím prvkem s pamětí až na 162 otisků. Přestože může být napájen až 5 V, datové piny jsou na 3,3V logice a jsou tak plně kompatibilní s mikrokontrolérem, se kterým komunikuje pomocí sběrnice UART.

Bezpečnostní parametr False Acceptance Rate (FAR) se vyjadřuje jako poměr počtu falešných akceptací k počtu pokusů o identifikaci, což udává podíl ověřovacích transakcí, při nichž jsou omylem potvrzeny nesprávné údaje totožnosti. FAR je klíčovým měřítkem pro hodnocení přesnosti a účinnosti biometrických bezpečnostních systémů, jako je software pro rozpoznávání obličeje nebo **skenery otisků prstů**. Nižší hodnota FAR znamená spolehlivější systém, zatímco vyšší hodnota FAR naznačuje vyšší riziko neoprávněného přístupu. Vybraný senzor DY50 dosahuje hodnoty < 0.001 %, díky této hodnotě lze senzor považovat za bezpečný.

Tabulka 2.4: Technické parametry senzoru DY50

Parametr	Hodnota
Napájecí napětí	3,6 – 6,0 V DC
Provozní proud	120 mA (max. 150 mA)
Komunikační rozhraní	UART
Kapacita úložiště	162 šablon
Doba snímání	< 1,0 s
Chybovost (FAR)	< 0,001 %
Pracovní teploty	-20 °C až +50 °C

2.4.3 Rotační enkodér

Pro interakci uživatele s grafickým rozhraním byl zvolen inkrementální rotační enkodér typu KY-040. Tento mechanický senzor převádí rotační pohyb hřídele na sérii digitálních pulsů, které jsou následně dekodovány mikrokontrolérem.

Princip funkce a zapojení

Modul KY-040 pracuje na principu kvadrurního kódování. Obsahuje dva spínače, které při otáčení generují dva fázově posunuté signály (označované jako CLK a DT). Sledováním fázového posuvu lze určit směr otáčení, zatímco počet pulsů určuje velikost úhlového posunutí. Enkodér také disponuje spínacím tlačítkem v ose hřídele. Toto umožňuje ovládat zařízení pomocí jednoho ovládacího prvku.

Tabulka 2.5: Technické parametry enkodéru KY040

Parametr	Hodnota
Pracovní napětí	3,6 – 5,25 V DC
Počet pulzů na otáčku	20
Životnost	30 000 otáček



(a) : OLED displej 1,3"



(b) : Senzor otisků prstů DY50



(c) : Rotační enkodér s integrovaným tlačítkem KY-040

Obrázek 2.7: Vybrané komponenty

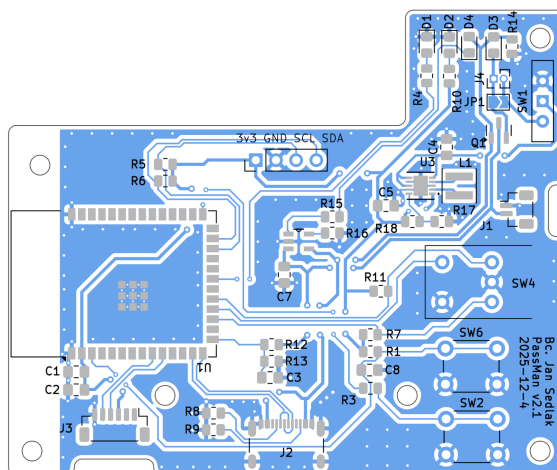
2.5 Návrh layoutu

Po vytvoření schématu zapojení, výběru vhodných komponent a ověření funkčnosti obvodu pomocí prototypu sestaveného z evaluačních desek bylo přistoupeno k návrhu vlastní desky plošných spojů (PCB). Tento krok umožnil optimalizovat výsledné řešení z hlediska rozměrů, elektrických vlastností i mechanické integrace do finálního zařízení.

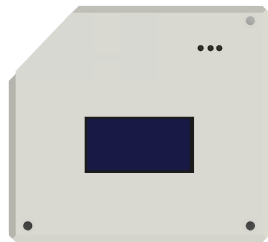
Při návrhu layoutu byl kladen důraz na dosažení kompaktních rozměrů desky, které vycházely z požadavku na snadnou manipulaci se zařízením a jeho ovládání. Návrh obrysu desky byl proveden s ohledem na umístění hlavních funkčních bloků, přičemž byly respektovány technologické limity výroby plošných spojů.

Mikrokontrolér s anténou byl umístěn na opačném konci oproti spínanému zdroji, aby se předešlo rušení spínanou frekvencí zdroje. Zároveň byla snaha, aby všechny napájecí komponenty byly v blízkosti a nebylo nutné vést napájecí cesty přes celé PCB. Rotační enkodér spolu s posuvným spínačem byl umístěn na pravé straně, aby odpovídal přirozenému úchopu zařízení a umožnil intuitivní ovládání bez nutnosti změny polohy ruky.

Zvláštní důraz byl kladen na umístění poměrně rozměrného senzoru otisků prstů. Jeho konektor byl umístěn na spodní straně desky vedle USB konektoru, avšak pro samotný senzor byl určen výkus PCB v horní části. Toto umístění zaručuje ergonomičnost a usnadňuje jeho používání. Samotné usazení je vidět v sekci 2.7.



Obrázek 2.8: Layout navrženého PCB



Obrázek 2.9: Pohled shora

■ 2.6 Výpočet spotřeby zařízení

■ 2.7 Konstrukční řešení a návrh krabičky

Cílem mechanického návrhu bylo vytvořit kompaktní a ergonomické řešení, které zajistí ochranu vnitřní elektroniky, fixaci komponent a komfortní ovládání zařízení jednou rukou. Jako výrobní technologie byl zvolen 3D tisk.

■ 2.7.1 Postup návrhu

Samotné řešení vycházelo z navrženého PCB popsaného v sekci 2.5.

■ 2.7.2 Vizualizace

Kapitola 3

Firmware

Tato kapitola se detailně věnuje návrhu a samotné implementaci firmwaru pro vyvinutého hardwarového správce hesel. Cílem bylo vytvořit stabilní a bezpečné prostředí, které zajišťuje veškerou logiku zařízení, od základní obsluhy databáze až po komunikaci s desktopovou aplikací.

Zvolený mikrokontrolér ESP32-S3 WROOM-1 disponuje vysokým výkonem a dostatečnou kapacitou paměti (SRAM i Flash) a integrovaným hardwarovým šifrovacím akcelerátorem, který byl pro vývoj bezpečnostního zařízení klíčový.

Vývoj firmwaru probíhal v programovacím jazyce C++. Využití tohoto jazyka umožnilo aplikovat objektivně orientovaný přístup (OOP), což přispělo k modularitě, čitelnosti a snadnější údržbě výsledného kódu. Projekt byl kompilován a spravován s využitím ekosystému PlatformIO pro MacOS.

Architektura firmwaru

Z hlediska softwarové architektury je firmware rozdělen do čtyř hlavních, na sobě nezávislých částí, což usnadňuje možnost úpravy a testování systému. Architektura se skládá z vrstvy správy dat, bezpečnostní vrstvy, komunikační vrstvy a uživatelského rozhraní.

3.1 Modul správy databáze

Jádrem firmwaru je systém obsluhující databázi hesel. Tento modul je zodpovědný za formátování dat, jejich bezpečné uložení do flash paměti zařízení a jejich následné načtení. Primárními operacemi jsou:

- Načtení databáze - Vždy po zapnutí zařízení automaticky, v rámci prvotního nastavení, načte databázi z flash paměti do operační paměti kontroléru pro potřeby zobrazení na displeji nebo pro odeslání do desktopové aplikace.
- Uložení databáze - Databáze je uložena pomocí souborového systému SPIFFS do flash paměti kontroléru.

- Smazání databáze - Funkce pro bezpečné odstranění celé databáze ze zařízení.

3.2 Komunikační rozhraní

Pro zajištění komunikace mezi hardwarovým správcem a desktopovou aplikací byl implementován vlastní komunikační protokol, viz sekce ???. Ke komunikaci je využíván sériový COM port.

Komunikační modul běží ve smyčce, kdy naslouchá příkazům z aplikace. Tyto požadavky jsou parsovány a na základě instrukcí (např. požadavek na načtení databáze ze zařízení, příkaz *load*) firmware provede příslušnou operaci v databázovém modulu a odešle zpět odpověď v podobě dat ze zařízení.

3.2.1 Návrh komunikačního protokolu

Komunikační protokol byl navržen pro zajištění komunikace mezi hardwarovým zařízením a desktopovou aplikací. Jedná se o textově orientovaný protokol fungující nad sériovým rozhraním. Po připojení k hostitelskému počítači se zařízení enumeruje jako sériový port (COM port). Komunikace probíhá asynchronně s přenosovou rychlostí 115200 baudů.

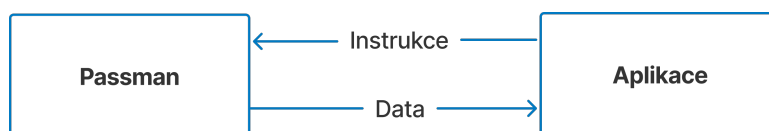
Inicializace relace (Handshake)

Protokol je navržen tak, aby po připojení obou komunikujících stran navázal spojení a přenesl informace o připojeném zařízení, jako je název zařízení, firmwarová verze, verze hardwaru a sériové číslo.

Tento krok slouží jako handshake, díky němu dokáže aplikace poznat, zda na komunikačním portu opravdu komunikuje kompatibilní zařízení.

Řízení toku dat

Po inicializačním procesu se z aplikace stává řídicí uzel (Master) a zařízení pasivně naslouchá na sériové lince a čeká na příchozí instrukce. V rámci naslouchání byla definována již výše zmíněná sada instrukcí.



Obrázek 3.1: Master-Slave architektura

Instrukce: info

Tok dat: Zařízení → Aplikace

Popis operace: Identifikace zařízení: Zařízení opětovně vyšle své identifikační informace.

Instrukce: load

Tok dat: Zařízení → Aplikace

Popis operace: Export databáze: Zařízení vyšle zašifrovanou databázi hesel.

Instrukce: download

Tok dat: Aplikace → Zařízení

Popis operace: Import databáze: Aplikace vyšle databázi do zařízení, které na základě instrukcí ukládá dešifrovaná data.

Instrukce: end

Tok dat: lokální akce

Popis operace: Ukončení relace: Přijetím této instrukce se aplikace odpojí z komunikačního portu zařízení a zařízení se vrátí do hlavní nabídky.

3.3 Bezpečnostní modul a ochrana dat

Bezpečnostní modul, vzhledem k primárnímu účelu zařízení, je gró celého firmwaru. Tento modul nese zodpovědnost za autorizaci uživatele a integritu dat. Bezpečnostní systém je rozdělen na dvě hlavní oblasti: autentizaci uživatele a kryptografické operace.

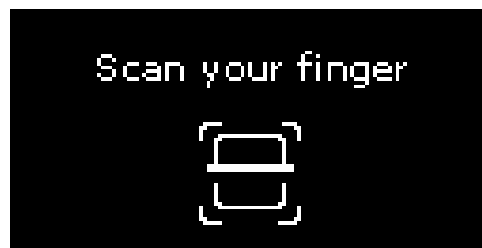
3.3.1 Autentizace uživatele

Zařízení je zabezpečeno dvěma způsoby autentizace: otiskem prstu a čtyřmístným číselným pinem. Ihned po spuštění zařízení je uživatel požádán o přiložení otisku prstu. V případě třetího zamítnutí přístupu má uživatel možnost provést autentifikaci pomocí pinu, na který jsou další dva pokusy. Proti útokům hrubou silou (Brute-force) je systém chráněn zablokováním po pátém, respektive druhém neúspěšném pokusu o zadání PIN kódu. V tomto případě se zařízení zablokuje na dobu třiceti sekund.

Proces odemknutí zařízení je implementován jako stavový automat sledující *unlockHandler*. Výchozí hodnota po spuštění je nastavena na 0, tedy uzamčený stav. Přejchod do odemčeného stavu je podmíněn úspěšnou validací uživatele otiskem prstu či zadáním správného pin kódu. Po úspěšné autentizaci je zařízení uživateli odemčeno.

V rámci autentizace uživatele byla implementována podpora uložení až 10 unikátních otisků prstů v rámci jednoho zařízení. Tato možnost zvyšuje komfort i flexibilitu používání zařízení. Databáze otisků prstů je uložena v paměti samotného senzoru.

- **Obouruční ovládání** - Umožňuje odemknout zařízení palcem i ukazováčkem obou rukou, což oceníte v situacích, kdy máte jednu ruku plnou.



Obrázek 3.2: Otisk prstu



Obrázek 3.3: UI: Přidání nového otisku prstu

- **Sdílený přístup** - V rámci rodinného nebo firemního použití lze udělit přístup více důvěryhodným osobám bez nutnosti sdílet PIN kód.

■ 3.3.2 Šifrování

K ochraně samotné databáze hesel ve flash paměti byla použita symetrická kryptografie AES-128. Použití vybraného kontroléru v tomto ohledu přináší významnou výhodu ve formě hardwarové akcelerace kryptografických funkcí. Díky tomu šifrování a dešifrování nezatěžuje hlavní procesor a je sníženo riziko úniku či ztráty citlivých dat přes postranní kanály (side-channel attacks). Klíč pro šifrování je uložen v paměti eFuse, která je dedikovaná k ukládání šifrovacích klíčů.

■ 3.3.3 Distribuce a správa bezpečnostního klíče

Vzhledem k implementaci symetrického šifrování je zapotřebí přenést symetrický šifrovací klíč mezi aplikací a zařízením. Přenos klíče probíhá při prvotní inicializaci zařízení. Níže je popsán celý proces od generování symetrického klíče až po jeho přenos do zařízení.

Generování šifrovacího klíče

Celý proces začíná v terminálu počítače. Uživatel si vygeneruje symetrický klíč, který si uloží do svého počítače. Pro generování lze využít například program *openssl* a příkaz *openssl rand 128 > symkeyfile.key*.

Načtení klíče do aplikace a jeho přenos

Při každém spuštění aplikace je klíč nahrán z uživatelského počítače do proměnné *secretKey* v aplikaci. V rámci prvotní inicializace zařízení je klíč přenesen do zařízení, kde je následně uložen do dedikované paměti eFuse. V tuto chvíli disponují obě strany stejným klíčem a komunikace může probíhat šifrovaně.

Tabulka 3.1: Placeholder Caption

Fáze	Entita	Akce
Generování	Terminál (PC)	Vygenerování K_s (Secret Key) a uložení na disk.
Inicializace	Aplikace → Zařízení	Jednorázový export klíče do paměti zařízení.
Provoz	Zařízení	Trvalé uložení v bezpečné oblasti paměti.
Runtime	Obě strany	Použití klíče pro šifrování relace (AES-GCM/CBC).

3.4 Řídící algoritmus

Hlavní řídicí logika programu je strukturována jako stavový automat (FSM). Tato implementace zajišťuje deterministické chování a zaručuje, že se zařízení nedostane do nedefinovaného stavu. Přechody mezi jednotlivými stavy jsou iniciovány na základě stavu ovládacího prvku, rotačního enkodéru. Díky stavovému automatu je možné snáze rozšířit chování v případě potřeby o další stav.

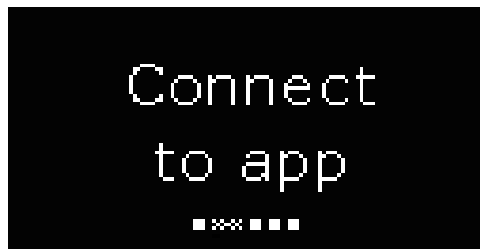
Po úspěšné autentizaci uživatele přivítá hlavní nabídka. Hlavní menu slouží jako rozcestník stavového automatu, na základě kterého zařízení přechází do následujících operačních režimů:

- **Database** - Zobrazí databázi hesel uložených na zařízení.



Obrázek 3.4: UI: Database

- **Connect to App** - Po úspěšné inicializaci relace s aplikací se přepne do režimu slave a naslouchá instrukcím z aplikace.



Obrázek 3.5: UI: Connect to Device

- **Save Database** - Uloží aktuální stav databáze do paměti zařízení.



Obrázek 3.6: UI: Save Database

- **Delete Database** - Vymaže databázi z paměti zařízení.

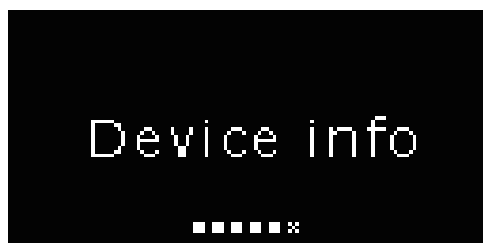


Obrázek 3.7: UI: Delete Database

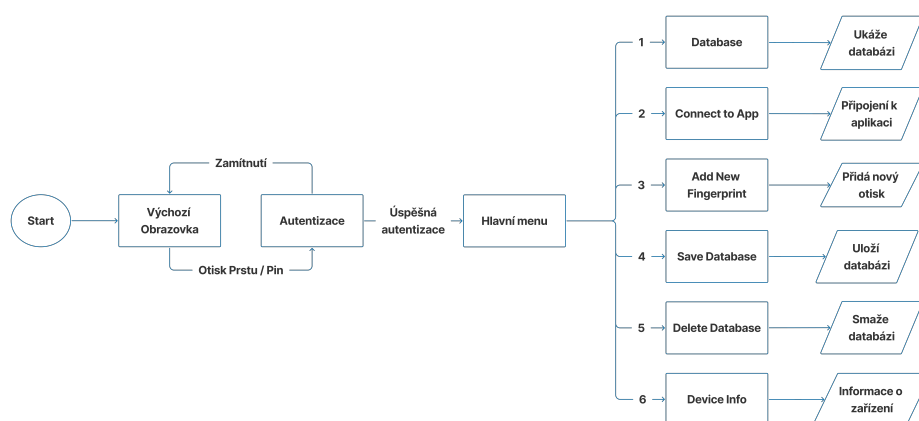
- **Device Info** - Zobrazí základní informace o zařízení a jeho verzi.

Celou logiku programu lze vidět na obrázku 3.9.

■ 3.5 Optimalizace a specifická řešení???



Obrázek 3.8: UI: Device Info



Obrázek 3.9: Diagram logiky firmwaru

Kapitola 4

Desktopová aplikace

Tato kapitola se zabývá vývojem desktopové aplikace, která slouží jako rozhraní mezi zařízením a uživatelem. Jejím hlavním účelem je zprostředkovat bezpečnou komunikaci se zařízením a umožnit správu uložených dat na zařízení.

Před samotným návrhem byly definovány základní pilíře, na kterých byl postaven vývoj aplikace. Jedná se o následující body:

- Bezpečnost - Komunikace mezi zařízením a aplikací musí být šifrovaná.
- Jednoduchost - Aplikace by měla být přehledná a jednoduchá na používání.
- Kompatibilita - Program je přenosný na operační systémy MacOS, Linux a Windows.

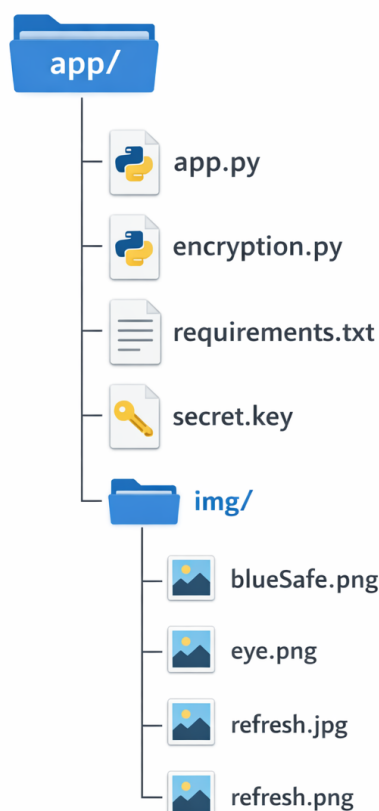
Aplikace byla implementována v jazyce Python s využitím frameworku PySide6. Cílovou platformou je operační systém MacOS (Tahoe 26.2), na kterém byla aplikace vyvíjena.

4.1 Architektura a datový model

Aplikace je postavena na monolitické architektuře bez separace do vrstev. Řešení uživatelského rozhraní, logiky aplikace, vstupy a výstupy jsou převážně řešeny v jedné třídě *MainWindow* v hlavním souboru *app.py*. Na obrázku 4.1 je znázorněna struktura projektu pro aplikaci.

Jak lze vidět na obrázku struktury, aplikace se skládá ze dvou skriptů: hlavního *app.py*, který řeší veškerou vizuální a funkční stránku aplikace, kromě šifrování, kdy *encryption.py* zajišťuje veškerou problematiku týkající se kryptografie.

Datový model pro ukládání bezpečnostních údajů z připojeného zařízení je řešen za pomoci tří paralelních seznamů (*entryID*, *entryUSERNAME*, *entryPSW*). Zobrazení je realizováno třemi widgety *QListWidget*, které korespondují s



Obrázek 4.1: Struktura aplikace

názvy seznamů, tedy ID, username a password. Synchronizace je indexová, tj. položky jsou svázány stejnou pozicí v jednotlivých seznamech. Tento přístup je jednoduchý a odpadá nutnost vytvářet externí databázi; je však citlivý na desynchronizaci při chybných operacích. Před nahráním jednotlivých seznamů do zařízení je databáze přeindexována počínaje $ID = 1$ až $ID = DB_{len}$, aby indexy korespondovaly s indexy v zařízení.

Informace o připojeném zařízení jsou ukládány ve formě *stringu*(deviceName, fwVersion, hwVersion, serialNumber). Údaje jsou následně zobrazovány ve widgetu s informacemi o připojeném zařízení.

4.2 Kybernetická bezpečnost

Přenos citlivých uživatelských dat je šifrován pomocí AES-128 v režimu CBC se samostatným IV (16 B). IV je prependovaný k šifrovanému textu a výsledek je ve formátu Base64. Šifrování je detailně popsáno v sekci 1.5 teoretické části.

Vzhledem k principu použité komunikace se dešifrování provádí při načítání databáze ze zařízení; v opačném případě aplikace provádí šifrování. Pro

vytvoření obou funkcí byl použit modul *Crypto.Cipher*, ze kterého bylo importováno *AES*. Implementaci funkcí pro zašifrování a dešifrování lze najít ve skriptu *encryption.py*.

4.3 Uživatelské rozhraní

Jak již bylo zmíněno v úvodu této kapitoly, k vytvoření aplikace byl použit framework PySide6, díky kterému je možné vytvořit uživatelsky přívětivé rozhraní.

Hlavní okno je v aktuální verzi fixováno na velikost 1050 x 520 pixelů a je rozděleno do tří hlavních bloků:

4.3.1 Connection tab

První sekcí uživatelského rozhraní je connection tab, který slouží k obsluze připojení zařízení k aplikaci pomocí COM portu. Uživatel má k dispozici seznam dostupných portů. Pokud je aplikace spuštěna dříve, než je fyzicky připojeno externí zařízení, je možné pomocí tlačítka *Search COMs* znovu vyhledat nově dostupné porty, které se objeví na konci seznamu. Pro připojení a odpojení slouží tlačítka *Connect*, resp. *Disconnect*. Baud rate pro komunikaci je nastaven na 115200 *Bd*. Posledním elementem je tlačítko s ikonou oka, které slouží pro zakrytí nebo odkrytí hesel v databázi.



(a) : Připojovací sekce - nepřipojené zařízení



(b) : Připojovací sekce - připojené zařízení

Obrázek 4.2: Připojovací sekce aplikace

4.3.2 Databáze

Hlavním prvkem celé aplikace je samotná databáze, která zabírá většinu okna. Databáze je v aplikaci uložena, jak již bylo zmíněno v sekci 4.1, do tří seznamů: *id*, *username*, *password*. Pro každý seznam je v aplikaci samostatný widget, jehož název koresponduje s názvy seznamů. Stejně jako databáze je řazena podle ID, je i zobrazení řazeno vzestupně od nejmenšího dostupného ID. Identifikátor není možné libovolně přepisovat.

4.3.3 Database tools tab

Poslední sekcí aplikace jsou dostupné uživatelské nástroje pro správu databáze. Widget *Device information* zobrazuje načtené informace o připojeném

ID:	USERNAME:	PASSWORD:
1	heslo1	
2	druidheslo	
3	Jenicek	
4	sedlajsi	
5	nahodneslo	
6	vygenerovaneslo	

ID:	USERNAME:	PASSWORD:
1	heslo1	sdb45jkh1a8
2	druidheslo	tsd4jdrhdhcs18
3	Jenicek	Jenicekheslo18
4	sedlajsi	vanabaling
5	nahodneslo	sa18a1a1bwa..sft
6	vygenerovaneslo	Beckl7FDnw

(a) : Odkrytá hesla
(b) : Skrytá hesla

Obrázek 4.3: Databáze hesel

zařízení, které jsou přenášeny při inicializaci spojení se zařízením. Jedná se o informace, jako jsou název zařízení, verze firmwaru a hardwaru, jeho sériové číslo a status připojení - *connected/disconnected*. V případě potřeby lze informace o zařízení aktualizovat pomocí příslušného tlačítka se šipkami.

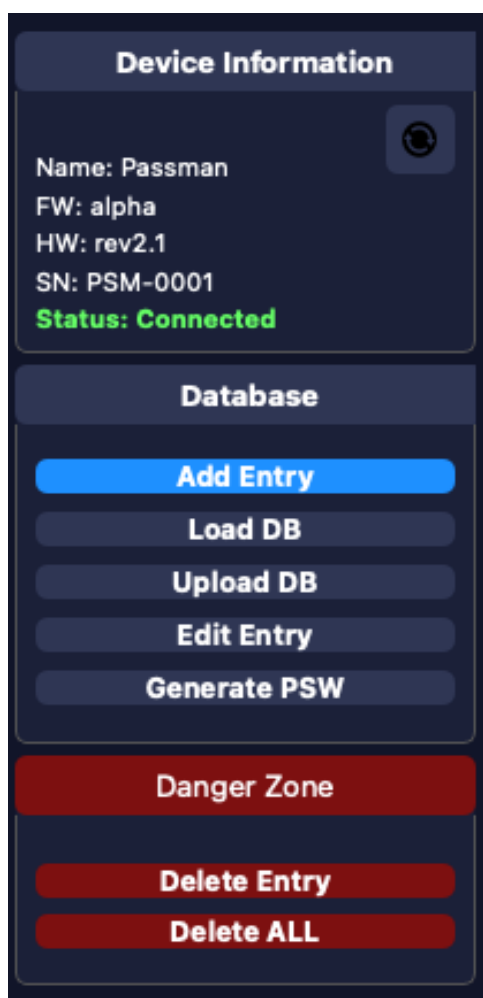
V sekci *Database* jsou k dispozici následující nástroje:

- Add Entry – Slouží k přidání nového hesla do databáze.
- Load DB – Dvojklikem načte data ze zařízení do databáze v aplikaci.
- Upload DB – Slouží k nahrání upravené databáze do zařízení.
- Edit Entry – Po vybrání záznamu z databáze umožní změnit *username* a *password*.
- Generate PSW – Generátor náhodného hesla.

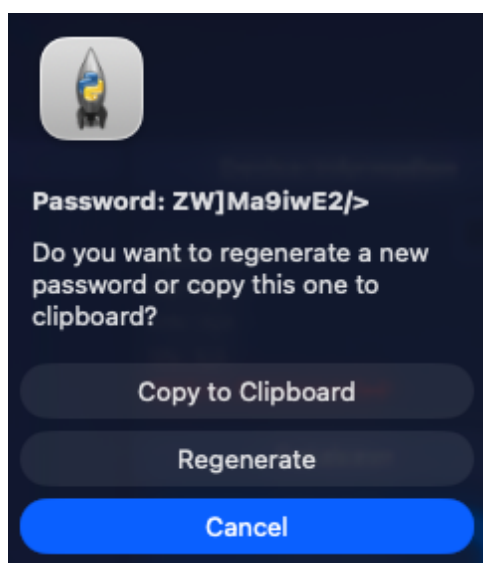
Posledním widgetem je *Danger Zone*, který obsahuje dvě tlačítka: *Delete Entry* pro smazání vybraného záznamu a *Delete ALL*, které smaže celou databázi. Po smazání celé databáze se hesla smaží pouze v aplikaci. Nahrát je zpět do aplikace lze použitím tlačítka *Load DB*. Pro definitivní smazání databáze i ze zařízení je třeba nahrát prázdnou databázi nebo provést smazání přímo v zařízení.

Generate PSW

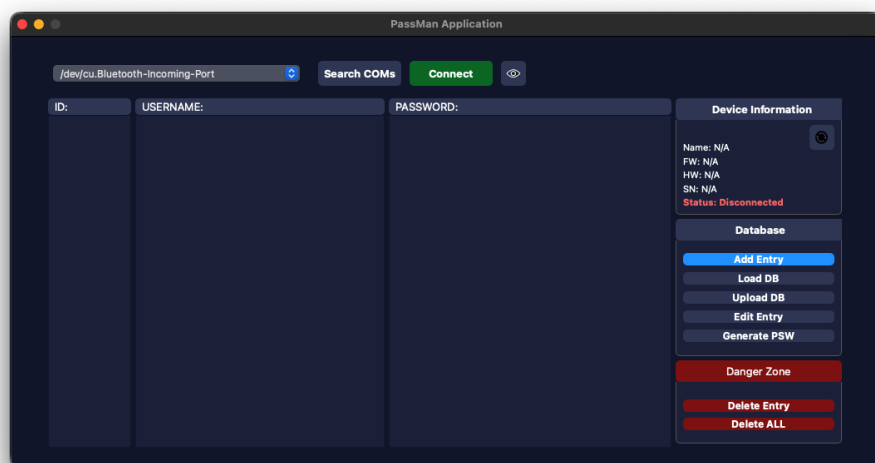
Databáze má vlastní nástroj pro generování náhodného hesla, který je nastaven dle zásad pro generování neprolomitelného hesla, viz sekce 1.1. Po kliknutí je uživatel vyzván k zadání délky generovaného hesla, poté nástroj heslo vygeneruje a v rámci pop-up zprávy zobrazí výsledek. Uživatel má možnost heslo zkopírovat do schránky nebo vygenerovat nové.



Obrázek 4.4: Nástroje



Obrázek 4.5: Funkce generování hesla



Obrázek 4.6: Uživatelské rozhraní aplikace



Kapitola 5

Ověření funkčnosti



Kapitola 6

Závěr



AES Advanced Encryption Standard. 9

ECC Error Checking and Correcting. 9

FAR False Acceptance Rate. 27

FSM Finite State Machine. 35

HSM Hardware Security Module. viii, 9, 10

I2C Inter-Integrated Circuit. 27

IoT Internet of Things. 12, 16

LDO Low Dropout Regulator. 24, 25

MFA Multi-Factor Authentication. 5

MMU Memory Management Unit. 12

MPU Memory Protection Unit. 12

OOP Object-Oriented Programming. 31

OTA Over the Air. 12

PCB Printed Circuit Board. 25, 29, 30

ROM Read-Only Memory. 9

RoT Root of Trust. 9

RSA Rivest–Shamir–Adleman. 9

SoC System on Chip. 11

TEE Trusted Execution Environment. viii, 10, 11

TOTP Time-based One-Time Password. 5

TPM Trusted Platform Module. viii, 9, 10

UART Universal Asynchronous Receiver-Transmitter. 27

USB Universal Serial Bus. 22, 23, 29

UX User Experience. 6



Literatura

- [1] How secure is my password?
- [2] Ross Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems*. John Wiley & Sons, 3rd edition, 2020.
- [3] Yifeng Cao, Ashutosh Dhekne, and Mostafa Ammar. Uwb-auth: A uwb-based two factor authentication platform. In *Proceedings of the 17th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '24)*, pages 27–38. ACM, 2024.
- [4] David Challener, Kent Yoder, Ryan Catherman, David Safford, and Leendert Van Doorn. *A Practical Guide to Trusted Computing*. IBM Press, 2008. Detailní rozbor TPM architektury.
- [5] David Cooper, William Polk, Andrew Regenscheid, and Murugiah Souppaya. Bios integrity measurement guidelines. Technical Report NIST SP 800-147, National Institute of Standards and Technology (NIST), 2011.
- [6] Paul A Grassi, Michael E Garcia, and James L Fenton. Digital identity guidelines: Authentication and lifecycle management. Technical Report Special Publication (SP) 800-63B, National Institute of Standards and Technology (NIST), 2017.
- [7] Joshua Gray, Virginia NL Franqueira, and Yijun Yu. Forensically-sound analysis of security risks of using local password managers. In *2016 IEEE 24th International Requirements Engineering Conference Workshops (REW)*, pages 114–121. IEEE, 2016.
- [8] Anil K Jain, Karthik Nandakumar, and Arun Ross. 50 years of biometric research: Accomplishments, challenges, and opportunities. *Pattern Recognition Letters*, 79:80–105, 2016.
- [9] JetBrains. Working with STM32 ARM TrustZone in CLion. JetBrains Blog, March 2026. [Online; navštíveno 10. 4. 2026].
- [10] Praveen Joshi. Common attacks on embedded systems and how to prevent them.

- [11] David Kleidermacher and Mark Kleidermacher. *Embedded Systems Security: Practical Methods for Safe and Secure Software and Communication Development*. Elsevier Science, Amsterdam, 2012.
- [12] Davide Maltoni, Dario Maio, Anil K Jain, and Salil Prabhakar. *Handbook of Fingerprint Recognition*. Springer Science & Business Media, 2nd edition, 2009.
- [13] National Institute of Standards and Technology. Digital identity guidelines: Authentication and lifecycle management. Technical report, NIST Special Publication 800-63B, Gaithersburg, MD, 2017.
- [14] National Institute of Standards and Technology. Fips pub 140-3: Security requirements for cryptographic modules. Technical report, U.S. Department of Commerce, 2019. Standard pro ochranu klíčů v HSM.
- [15] Sandro Pinto and Nuno Oliveira. Demystifying arm trustzone: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 51(6):1–36, 2019.
- [16] Ariel Piper. Unbreakable passwords hackers can’t crack.
- [17] Srivaths Ravi, Anand Raghunathan, Paul Kocher, and Sunil Hattangady. Security in embedded systems: Design challenges. In *ACM Transactions on Embedded Computing Systems (TECS)*, volume 3, pages 461–491. ACM, 2004.
- [18] Bruce Schneier. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. John Wiley & Sons, 20th anniversary edition edition, 2015.
- [19] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley & Sons, 10th edition, 2018. Teoretický základ pro MMU a virtuální paměť.
- [20] William Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 7th edition, 2017.
- [21] William Stallings and Lawrie Brown. *Computer Security: Principles and Practice*. Pearson, 4th edition, 2018. Kapitoly zaměřené na Hardware Security a Trusted Computing.
- [22] Stephan Electronics. Tindie: Mooltipass mini ble authenticator, 2024. Online; citováno 2024-05-01.
- [23] Joseph Yiu. *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*. Newnes, 3rd edition, 2013. Klíčový zdroj pro pochopení mechanismů MPU u ARM procesorů.